

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #10 - Semana 11 – Sivarama - Capítulo 18
Aritmética con representaciones ASCII y BCD

Estudiante: Mauricio González Prendas – Carné: 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 07/10/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 379–390.

Abstract—*Arithmetic operations using ASCII and BCD (Binary-Coded Decimal) offer alternatives to the binary format, reducing conversion overhead when handling numbers in non-binary forms, such as in financial applications. In the ASCII representation, numbers are stored as character strings, while BCD can be unpacked (one digit per byte) or packed (two digits per byte). Key instructions like AAA, AAS, DAA, and DAS are used to adjust arithmetic results in these formats. The trade-offs between efficiency and processing overhead in binary, ASCII, and BCD representations highlight the appropriate use cases for each method.*

I. Aritmética con representaciones ASCII y BCD

En la mayoría de los sistemas, los datos numéricos se procesan en formato binario. Sin embargo, existen dos representaciones alternativas: **ASCII** y **BCD** (Decimal Codificado en Binario), que reducen la necesidad de conversión. Aunque estas representaciones evitan la sobrecarga de conversión, el procesamiento de números en estos formatos es más lento en comparación con el binario. Este capítulo analiza cómo realizar operaciones aritméticas en estas representaciones y las compensaciones entre la sobrecarga de conversión y el tiempo de procesamiento.

II. Representación de números en ASCII y BCD

A. Representación en ASCII

Los números se almacenan como cadenas de caracteres ASCII. Por ejemplo, el número 1234 se representa como 31 32 33 34H, donde cada byte contiene el código ASCII de cada dígito. Las operaciones aritméticas requieren ajustes especiales para mantener la coherencia en las operaciones.

B. Representación en BCD

El BCD puede representarse de dos maneras:

1. **BCD desempquetado**: Cada dígito decimal se almacena en un byte, similar a la representación ASCII, pero con su valor binario correspondiente. Ejemplo: 1234 es 01 02 03 04H.
2. **BCD empaquetado**: Dos dígitos se almacenan en un solo byte, lo que reduce a la mitad el espacio de almacenamiento. Ejemplo: 1234 es 12 34H.

III. Operaciones aritméticas en ASCII

A. Instrucción AAA (Ajuste ASCII después de la suma)

La instrucción **AAA** ajusta el resultado de una suma para convertirlo a una representación BCD desempquetada. Si los cuatro bits menos significativos de **AL** son mayores que 9 o si el indicador auxiliar está activado, **AAA** añade 6 a **AL** y ajusta los indicadores de acarreo.

Ejemplo:

```
mov AL, '6' ; AL = 36H
add AL, '7' ; AL = 6DH (incorrecto)
aaa          ; AL = 03H, ajusta a BCD
```

B. Instrucción AAS (Ajuste ASCII después de la resta)

AAS ajusta el resultado de una resta similar a **AAA**. Si los cuatro bits menos significativos de **AL** son mayores que 9 o si el indicador auxiliar está activado, **AAS** resta 6 de **AL**.

C. Instrucción AAM (Ajuste ASCII después de la multiplicación)

AAM ajusta el resultado de una multiplicación, dividiendo el valor en **AL** entre 10. El cociente se almacena en **AH** y el resto en **AL**.

Ejemplo:

```
mov AX, 0207H ; AX = 0207H (desempaquetado BCD)
aam           ; AX = 0107H
```

D. Instrucción AAD (Ajuste ASCII antes de la división)

AAD prepara el divisor en **AX** antes de realizar una división de números desempquetados. Multiplica **AH** por 10 y suma el valor a **AL**.

Ejemplo:

```
mov AX, 0207H ; AX = 0207H
aad           ; AX = 001BH (preparado para dividir)
```

IV. Operaciones aritméticas en BCD empaquetado

A. Instrucción DAA (Ajuste decimal después de la suma)

DAA ajusta el resultado de una suma para asegurar la representación correcta en BCD empaquetado. Si los cuatro bits menos significativos de **AL** son mayores que 9 o si hay un acarreo, se suma 6 a **AL**.

Ejemplo:

```
mov AL, 29H ; AL = 29H
add AL, 69H ; AL = 92H (incorrecto)
daa          ; AL = 98H (ajustado)
```

B. Instrucción DAS (Ajuste decimal después de la resta)

DAS ajusta el resultado de una resta para conformarse al formato BCD empaquetado. Si los cuatro bits menos significativos de **AL** son mayores que 9 o si hay un acarreo, resta 6 de **AL**.

Ejemplo:

```
mov AL, 71H ; AL = 71H
sub AL, 43H ; AL = 2EH (incorrecto)
das          ; AL = 28H (ajustado)
```

V. Comparación entre aritmética binaria, ASCII y BCD

El formato binario es el más eficiente en términos de espacio y procesamiento. Sin embargo, cuando los datos de entrada y salida están en ASCII, puede ser más eficiente evitar la conversión y procesar directamente en ASCII o BCD. Aunque las operaciones en binario son más rápidas, las representaciones en ASCII y BCD pueden ser útiles en aplicaciones específicas, como la manipulación de datos financieros donde los errores de redondeo son críticos.